



Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera: Graph-Based ML Pipelines, Benchmarking Platform, and Multi-Target Model Deployment

Pipeline Parallelism, Web-Based Leaderboards, and Containerized
Deployments



A Graduation Project Individual Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mazen Adel
Student ID: 9220625

Supervised by

Dr. Salma Abdel Monem

June 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the author or department.

Abstract

This report details the individual design, implementation, and verification of three core modules in the Accelera framework: the Pipeline Module (graph execution and parallelism), the Benchmark Module (REST API, authentication, dashboards, and automated scoring), and the Deployment Module (multi-target deployment, runtime schema validation, telemetry logging, and model version control).

Within the Pipeline Module, Mazen implemented the C++ back-end execution engine supporting branching workflows, topological sorting, DAG constraints, and duplicate-node optimization. Specifically, this work presents his implementation of a thread-safe parallel scheduler using counting semaphores that executes independent branches concurrently while releasing consumed intermediate buffers to minimize peak memory usage, alongside a native Merge Node implementing class and probability-based majority voting strategies.

Within the Benchmark Module, Mazen developed a Node.js REST API using Express and MongoDB to handle secure user authentication (JWT and bcrypt), dataset uploads, leaderboard management, and validation of submitted predictions against hidden ground truths. He also built a React frontend consisting of dedicated User and Admin Dashboards for task filtering, leaderboard viewing, and administrator-level user management.

Finally, the Deployment Module forms the main contribution of this individual work. It integrates a production-ready FastAPI web server featuring runtime data validation via Great Expectations, an asynchronous request tracking and telemetry logging database, a lightweight model Version Control System (VCS) with SHA-1 hash commits and automatic rollback, and an automated deployment engine targeting Local Docker, Heroku, and remote AWS EC2 instances over SSH.

Contents

1	Introduction	3
1.1	Project Context and Goals	3
1.2	Mazen Adel's Contributions	3
1.3	Report Organization	3
2	Literature Survey	5
2.1	Machine Learning Pipeline Abstractions	5
2.2	Automatic Concurrency and GIL Release	5
2.3	Web Serving and Data Quality Layers	5
2.4	Multi-Target Deployment Platforms	5
3	System Design and Architecture	6
3.1	Pipeline Module Architecture	6
3.2	Benchmark Module Architecture	6
3.2.1	Database Schema Design	6
3.2.2	Dashboard Views	7
3.3	Deployment Module Architecture	7
4	Implementation Details	9
4.1	Pipeline Module Parallel Scheduler	9
4.1.1	Execution Flow and GIL Release Implementation	9
4.1.2	Concurrency Semaphores and GPU Serialization	10
4.2	Merge Node and Voting Strategies	10
4.2.1	Voting Strategy Implementation and Edge Cases	10
4.3	Benchmark REST APIs and Dashboards	11
4.3.1	Secure Backend Authentication Routes	11
4.3.2	Submission Processing and Score Validation Route	11
4.3.3	Frontend Dashboards	11
4.4	Deployment Module serving and Validation	12
4.4.1	Model Serving Subsystem	12
4.4.2	Runtime Schema Validation	12
4.4.3	Telemetry and Request Tracking	13
4.5	Multi-Target Deployment Orchestration	13
4.6	Model Version Control (VCS)	14
4.7	Command-Line Interface and CLI Commands	15
4.8	Design Constraints and Challenges	16
5	System Testing and Verification	18
5.1	Pipeline Verification	18

5.2	Benchmark Subsystem Verification	18
5.3	Deployment Module Verification	18
5.3.1	Subsystem Verification Details	18
5.3.2	Deployment Test Results	19
6	Conclusions and Future Work	21
6.1	Summary of Contributions	21
6.2	Future Extensions	21

List of Figures

3.1	Optimization of Duplicate Node Sharing in DAGs	6
3.2	Access Control matrix for dashboards	7
3.3	Model Serving and Validation Architecture	8
4.1	Deployment Orchestration Workflow	14
4.2	VCS Commit Workflow	15
4.3	VCS Deploy Workflow	15

List of Tables

5.1	Deployment Module Test Suite Verification Summary	20
-----	---	----

Chapter 1: Introduction

1.1 Project Context and Goals

Developing and deploying machine learning pipelines in professional settings requires solving challenges that span different parts of the software lifecycle. A simple, linear script is often sufficient for training a single model on a local workstation. However, a production-grade machine learning system must evaluate multiple models, preprocessing methods, and hyperparameters, run experiments fairly, track version histories, and host models in scalable environments.

Accelera addresses these challenges by organizing the workflow into separate but integrated modules. This individual work focuses on three essential layers:

1. **The Execution Layer (Pipeline Module):** Runs multi-branch pipelines efficiently on multi-core machines, handling dependencies, branching, merging, and memory management.
2. **The Collaboration Layer (Benchmark Module):** Provides a central, secure platform where different developers can test their models on the same datasets under the same evaluation metrics.
3. **The Serving Layer (Deployment Module):** Acts as the main focus of this work, responsible for packaging trained models, validating input data, saving prediction histories, and pushing services to container platforms.

1.2 Mazen Adel's Contributions

As part of the Accelera development team, the author designed, implemented, and verified the following components:

- **Pipeline Module:** Developed the native C++ `MergeNode` class and voting algorithms. Built the concurrent graph runner (`runParallel`) using semaphores, GIL management, and memory optimizations.
- **Benchmark Module:** Programmed the Node.js Express backend and MongoDB schemas for users, benchmarks, and submissions. Designed the React dashboards for users and administrators.
- **Deployment Module:** Designed and built the complete model serving system, which includes the FastAPI server, Uvicorn server, Great Expectations validation layer, telemetry logger, and CLI deployment tools for Docker, Heroku, and AWS EC2. Created the custom Model Version Control (VCS) subsystem.

1.3 Report Organization

The rest of this report is organized as follows: Chapter 2 reviews the relevant background and related technologies. Chapter 3 describes the architecture, schemas, and design

constraints of the three modules. Chapter 4 explains the details of the implementation and algorithms. Chapter 5 presents the testing results and verification, and Chapter 6 concludes with future improvements.

Chapter 2: Literature Survey

2.1 Machine Learning Pipeline Abstractions

Machine learning pipelines are commonly used to organize workflows. The most popular pipeline system is the linear model in `scikit-learn`, which chains transformers and estimators together. However, a linear pipeline cannot naturally represent complex, non-linear workflows. For example, if a developer wants to compare three scaling methods and two classifiers, a linear system requires building six separate pipelines. This duplicates both code and execution time, as early steps must run repeatedly. Modern systems address this by representing workflows as Directed Acyclic Graphs (DAGs), where nodes represent computational steps and edges represent data dependencies.

2.2 Automatic Concurrency and GIL Release

In Python-based frameworks, running pipelines concurrently is limited by the Global Interpreter Lock (GIL). The GIL prevents multiple threads from executing Python bytecodes at the same time. To achieve true parallel execution on multi-core systems, heavy computations must be offloaded to compiled native languages like C++ using binding tools like `pybind11`. When executing C++ code, the Python GIL can be released (`py::gil_scoped_release`), allowing separate native threads to run independently on different CPU cores.

2.3 Web Serving and Data Quality Layers

Model serving requires turning saved binaries (such as pickle files) into active web endpoints. FastAPI is a common choice for this, utilizing asynchronous loops to handle concurrent requests. However, raw inputs submitted by users can contain invalid formats or values that cause models to crash. Data validation tools like Great Expectations help prevent this. They define schema rules (such as expected columns, types, and range boundaries) to check data quality at runtime before sending inputs to the model.

2.4 Multi-Target Deployment Platforms

Deploying serving containers involves different target architectures:

- **Local Docker:** Provides local isolation, ensuring the service runs identically in development and production.
- **Heroku Container Registry:** Simplifies cloud hosting by managing HTTP routing and container scaling automatically.
- **AWS EC2:** Offers raw virtual machines in the cloud, requiring custom SSH management, codebase synchronization, remote container building, and manual firewall configuration.

Chapter 3: System Design and Architecture

3.1 Pipeline Module Architecture

The Pipeline Module represents machine learning workflows as a DAG. Nodes are split into six functional categories: input, preprocessing, model, prediction, metric, and merge. The DAG is compiled by sorting the nodes topologically, which ensures that parent nodes execute before their child nodes.

Two major optimizations were introduced by Mazen to improve execution performance:

1. **Duplicate first-node sharing:** When multiple branches start with the same preprocessing step, the engine automatically shares the output of the first node rather than duplicating the calculation (Figures 3.1a and 3.1b).
2. **Active consumer counting:** The engine counts the active downstream consumers for each intermediate array. Once all child nodes finish execution, the parent node's temporary data is freed from RAM.

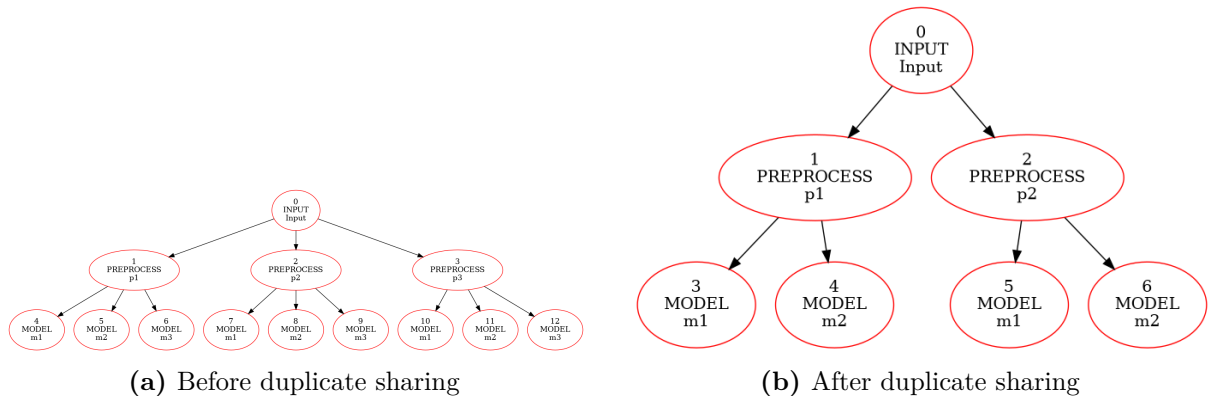


Figure 3.1: Optimization of Duplicate Node Sharing in DAGs

3.2 Benchmark Module Architecture

The Benchmark Module provides a web-based platform for comparing model performances. It consists of a React frontend and a Node.js backend using Express and MongoDB.

3.2.1 Database Schema Design

The database schemas are defined using Mongoose. The three main schemas are:

- **User Schema:** Stores names, emails, hashed passwords (via bcrypt), and role properties ("user" or "admin").
- **Benchmark Schema:** Defines the title, description, target column name, problem type, Google Drive dataset links, and a reference to the evaluation metric schema.

- **Submission Schema:** Tracks the submitted repository link, predictions file link, submission date, and calculated score.

3.2.2 Dashboard Views

The User Dashboard lists active benchmarks and submissions. The Admin Dashboard is restricted to administrative users (Figure 3.2), allowing them to browse and delete users, manage roles, and register new evaluation metrics.

Role	Dashboard Access	Authorized Operations
user	User Dashboard	View leaderboards, create benchmarks, submit predictions
admin	Admin Dashboard	View users, delete users, register new metrics

Figure 3.2: Access Control matrix for dashboards

3.3 Deployment Module Architecture

The Deployment Module is the primary contribution of this work. It packages trained estimators and preprocessors into a containerized REST service. Figure 3.3 outlines the serving and validation components.

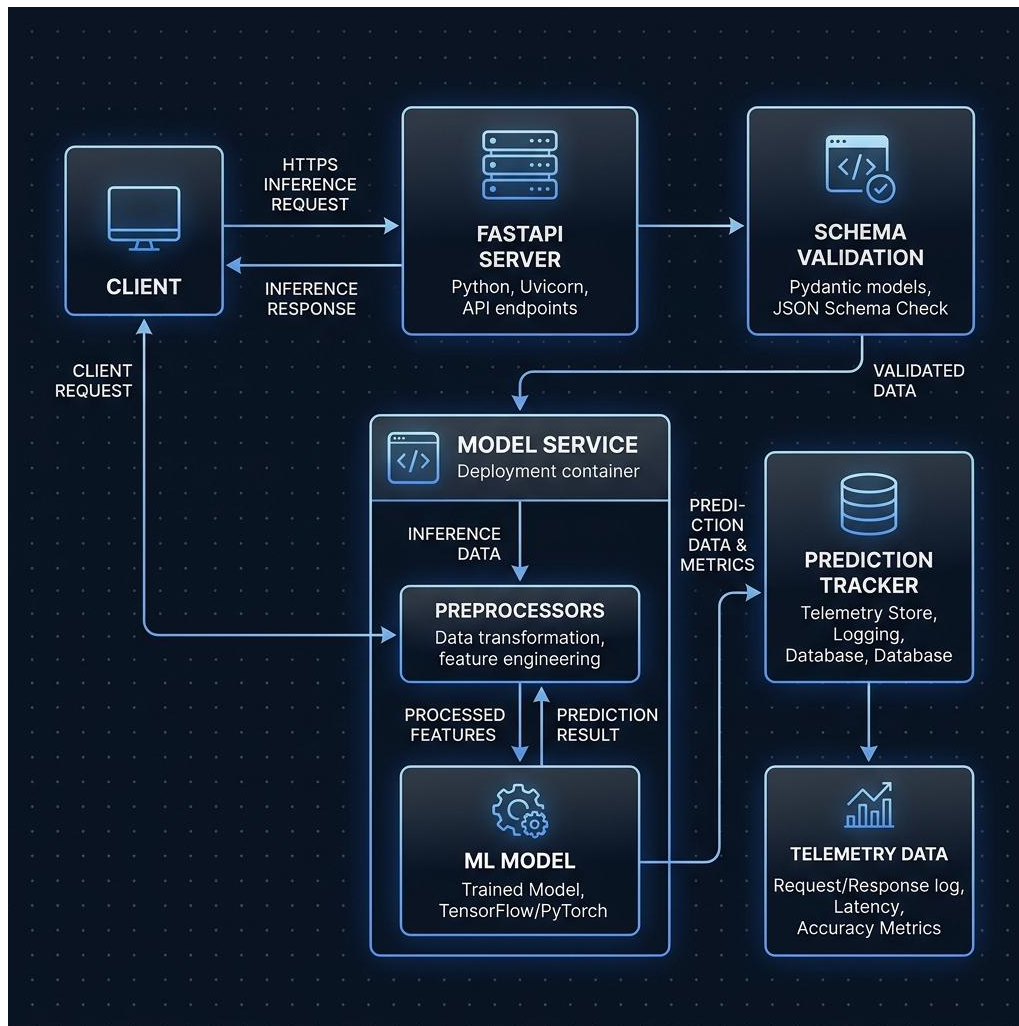


Figure 3.3: Model Serving and Validation Architecture

The system design relies on four core elements:

- **FastAPI Server:** Exposes standard inference endpoints for JSON payloads and batch inference endpoints for uploaded CSV files.
- **Great Expectations Schema Validation:** Validates incoming request values against expected bounds, preventing invalid inputs from reaching the model.
- **Telemetry Tracking:** Logs request inputs, outputs, timestamps, and execution times asynchronously to a JSON lines file on disk.
- **Model Version Control (VCS):** Creates unique commits of configuration and model binaries, allowing rollbacks to older deployments.

Chapter 4: Implementation Details

4.1 Pipeline Module Parallel Scheduler

The parallel scheduler in Accelera is implemented in C++ in `src/core/graph.cpp`. When parallel execution is enabled, the graph uses `std::async` to execute independent nodes concurrently. The scheduling logic uses counting semaphores to limit the number of active CPU threads, and a binary semaphore to serialize GPU access.

Algorithm 4.1 shows the parallel execution logic. It tracks finished and started nodes, release the GIL during execution, and catches exceptions thrown by separate threads.

Algorithm 4.1: Parallel Execution Engine in C++

```

1: Input: execution_order nodes, cpu_threads limit
2: Output: executed graph with updated node states
3:
4: 1: Release GIL via py::gil_scoped_release
5: 2: Initialize finished_executing map to false for all nodes
6: 3: Initialize started_executing map to false for all nodes
7: 4: Initialize CPU semaphore with cpu_threads count
8: 5: Initialize GPU semaphore with 1 (binary semaphore)
9: 6: Initialize atomic counter rem_nodes = size(execution_order)
10: 7: while rem_nodes > 0 do
11: 8:   Acquire nodes semaphore
12: 9:   for each node in execution_order do
13: 10:    if started_executing[node] == true then continue
14: 11:    check if all parent nodes are finished_executing
15: 12:    if parents_finished == true then
16: 13:      if node.uses_gpu == true then
17: 14:        Acquire GPU semaphore
18: 15:      else
19: 16:        Acquire CPU semaphore
20: 17:      end if
21: 18:      set started_executing[node] = true
22: 19:      spawn std::async(launch::async, [node]() {
23: 20:        Acquire GIL via py::gil_scoped_acquire
24: 21:        try {
25: 22:          node->execute()
26: 23:        } catch (...) {
27: 24:          store_exception()
28: 25:        }
29: 26:        decrement consumer counts for parent nodes
30: 27:        if parent consumer count == 0 then
31: 28:          release parent temporary memory
32: 29:        end if
33: 30:        set finished_executing[node] = true
34: 31:        decrement rem_nodes atomically
35: 32:        Release CPU/GPU semaphores
36: 33:        Release nodes semaphore
37: 34:      })
38: 35:    end if
39: 36:  end for
40: 37: end while
41: 38: Re-throw any caught exceptions to the Python runtime

```

4.1.1 Execution Flow and GIL Release Implementation

The primary implementation challenge in the parallel execution flow was GIL (Global Interpreter Lock) contention. Because Python objects (such as scikit-learn estimators) are executed inside separate worker threads, concurrent calls will block each other if they

attempt to manipulate Python references simultaneously. Mazen solved this by structuring a two-tier block wrapper in `src/core/graph.cpp`:

1. Before entering the loop-scheduling phase, the main thread releases the GIL using `py::gil_scoped_release`. This allows other native OS threads to run without waiting.
2. Within each spawned worker thread, before invoking the node's `execute()` function, the GIL is explicitly acquired using `py::gil_scoped_acquire`. As soon as the call exits, the scope ends, immediately freeing the GIL for other concurrent threads.

4.1.2 Concurrency Semaphores and GPU Serialization

To control the allocation of hardware resources, Mazen implemented custom semaphores using C++ synchronization primitives:

- **CPU Thread Limiter:** A counting semaphore initialized to `ACCELERA_NUM_THREADS` (or system cores minus one). Worker threads block on this semaphore before execution, preventing context-switching overhead from over-subscription.
- **GPU Access Lock:** A binary semaphore (mutex) used to serialize accesses to CUDA contexts. Because multiple threads submitting operations to the same GPU concurrently can cause driver crashes, any node with the `uses_gpu` flag is forced to acquire this binary lock, serializing GPU execution while keeping CPU-bound preprocessing branches running in parallel.

4.2 Merge Node and Voting Strategies

The `MergeNode` class combines outputs from multiple prediction branches. Mazen implemented this node in C++ (`src/nodes/merge.cpp`) and connected it to NumPy and SciPy components. The node supports a hard-voting strategy implemented in C++:

- **Direct Class Predictions (1D Arrays):** Collects predicted category labels from all upstream branches, stacks them into a single 2D NumPy array (rows correspond to samples, columns to model outputs), and uses `scipy.stats.mode` to extract the majority class label.
- **Probability Predictions (3D Arrays):** Stacks predicted probabilities across all branches. It computes class labels for each model using `np.argmax` across the probability dimensions and then applies majority voting on the resulting class labels.

4.2.1 Voting Strategy Implementation and Edge Cases

In `src/nodes/merge.cpp`, Mazen handled several implementation edge cases:

1. **Single-Model Fallback:** If the merge node only receives a single input branch, it directly routes the input to the output without calling voting functions, avoiding unnecessary computation.
2. **Tie-Breaking:** When the voting results are split evenly, the node falls back to selecting the output of the first model (the primary branch), guaranteeing deterministic results.

3. **GIL Safety in Numpy Stack:** Because stack operations call back into the Python/NumPy interpreter, the merge execution block is wrapped in a GIL acquisition scope, securing memory allocations.

4.3 Benchmark REST APIs and Dashboards

Mazen designed and built the full client-server subsystem for the Benchmark module.

4.3.1 Secure Backend Authentication Routes

Implemented in Express and MongoDB in `benchmark/backend/routes/user.js`:

- **Login Route (POST /login):** Sanitizes the input email, locates the user document, and compares the password against the stored bcrypt hash. Upon matching, it generates a signed JSON Web Token (JWT) with a 7-day expiration containing the user's MongoDB ID and role payload.
- **Signup Route (POST /signup):** Hashes new user passwords using a salt work factor of 10. If the email matches the environment parameter `ADMIN_EMAIL`, the server automatically promotes the role to "admin".
- **Deletion Route (DELETE /:id):** Restricted to admin users, allowing the deletion of any user profile except their own to prevent lockout.

4.3.2 Submission Processing and Score Validation Route

Implemented in `benchmark/backend/routes/submissions.js`, the scoring route runs the evaluation pipeline when a user submits a CSV file:

1. Downloader modules download the predictions CSV file and the hidden ground truth CSV from Google Drive.
2. The backend validates the integrity of the files, verifying that the number of rows matches.
3. It executes a Python worker subprocess configured with the target metric schema (e.g. F1-score or Mean Squared Error), calculates the performance score, and saves the submission to MongoDB.

4.3.3 Frontend Dashboards

Built in React (`benchmark/frontend/src/components/`) using state Hooks and async fetch requests:

- **User Dashboard (dashboard.jsx):** Displays a user's details, lists all benchmarks created by the user, and presents a grid of their submissions with performance scores and date history.
- **Admin Dashboard (adminDashboard.jsx):** Renders user lists and provides interactive deletion controls, checking JWT payloads to ensure administrative access.

4.4 Deployment Module serving and Validation

Mazen was fully responsible for implementing the Deployment module, which converts trained pipelines into containerized web services.

4.4.1 Model Serving Subsystem

Model serving is built on FastAPI and Uvicorn to handle concurrent HTTP requests. The subsystem loads the model and preprocessors via `ModelService`, which inspects the configuration (`config.json`) to locate the serialized artifacts. During execution, it runs input data through any fitted preprocessors in sequence before invoking the model's prediction method. A web-based graphical user interface (GUI) is dynamically rendered based on the expected input features, allowing users to submit single predictions interactively.

The following methods and files implement the serving logic:

- `modelservice.py`:
 - `ModelService.load()`: Parses `config.json` to discover the active model configuration. It opens and loads the main estimator file (e.g. `model.pkl`) and any auxiliary preprocessors (e.g. scalers or encoders) from the models directory.
 - `ModelService.predict(input_data, validate=True)`: Performs prediction. If validation is active, it calls `validate_input(input_data)` to receive validated and coerced values, converts the inputs into a NumPy array, adjusts dimensions, applies all loaded preprocessor transforms sequentially, and calls the underlying estimator's `predict()` method.
- `server.py`:
 - `_predict(input_data, endpoint, filename=None)`: Coordinates serving telemetry. It records starting timestamps using `time.perf_counter()`, executes model prediction via `ModelService.predict()`, calculates API response latencies, and logs metrics to the telemetry storage.
 - `/predict` and `/predict/csv` Endpoints: Standard HTTP POST methods. The first endpoint receives a structured JSON payload, while the second parses uploaded CSV files into Pandas DataFrames.
 - `/gui` and `/health` Endpoints: The GUI endpoint returns the dynamic interactive testing form via `_render_gui(schema)`, while the health endpoint serves load balancers with standard status checks.

4.4.2 Runtime Schema Validation

To prevent invalid inputs from causing model crashes or silent degradation, incoming payloads are validated against a schema specified in the configuration. The validation layer uses Great Expectations to construct an ephemeral data source context. Columns are validated for expected names, missing entries, unexpected inputs, and value bounds (e.g., minimum and maximum thresholds). Data types are coerced automatically (e.g., matching string, boolean, integer, or floating-point formats) to match the training format, raising a structured `SchemaValidationError` if coercion fails.

The schema validation system is implemented in `schema_validation.py`:

- `InputSchema.__init__(config)`: Extracts features and limits from the configuration. If schema validation is enabled, it requests a Great Expectations context (`gx.get_context()`), generates a unique UUID-based data source, registers a pandas-based DataFrame data asset, and configures a whole-dataframe batch definition.
- `InputSchema.validate(input_data)`: Normalizes the input payload into a DataFrame. It checks for expected column presence, detects extra or missing features, runs the Great Expectations expectation suite, and executes type coercion.
- `InputSchema.coerce_columns(df, errors)`: Loops through features to coerce types. It casts strings to numerical columns using `pd.to_numeric(..., errors='coerce')`, checks for integer constraints (`value % 1 == 0`), and processes boolean symbols (such as replacing `"true"/"false"` with Python booleans).
- `InputSchema.to_dataframe(input_data)`: Utility method to convert list, dict, or DataFrame structures into standard Pandas format.

4.4.3 Telemetry and Request Tracking

Production deployments capture inference requests and runtime metrics using a lightweight tracking layer. When enabled, the tracker logs prediction inputs, model outputs, processing latencies (measured in milliseconds), and timestamps. To avoid performance degradation on the prediction path, logging is written incrementally to a JSON lines (`.jsonl`) file.

Telemetry tracking is implemented in `tracking.py`:

- `PredictionTracker.__init__(config)`: Sets up the file system paths and creates/opens the telemetry log file.
- `PredictionTracker.log_prediction(inputs, outputs, latency, status)`: Formats request parameters, response targets, server latency, and datetime strings into a single-line JSON string and appends it to the logging stream.

4.5 Multi-Target Deployment Orchestration

The orchestration engine automates containerizing and launching the service across three distinct environments. It stages the codebase, copies project dependency requirements, and builds dockerized containers to ensure portability and reproducibility across platforms.

1. **Local Docker**: Generates a clean `Dockerfile` using a Python slim base image, stops any conflicting containers on the target port, builds the container image locally, and runs it.
2. **Heroku**: Authenticates with the Heroku CLI and pushes/releases the Docker container using Heroku's Container Registry.
3. **AWS EC2**: Creates an SSH transport connection and synchronizes the staging directory to the target host using `rsync` (excluding Python cache directories). It installs and starts the Docker daemon, builds the container on the instance, and executes local health check probes before verifying the public web endpoint.

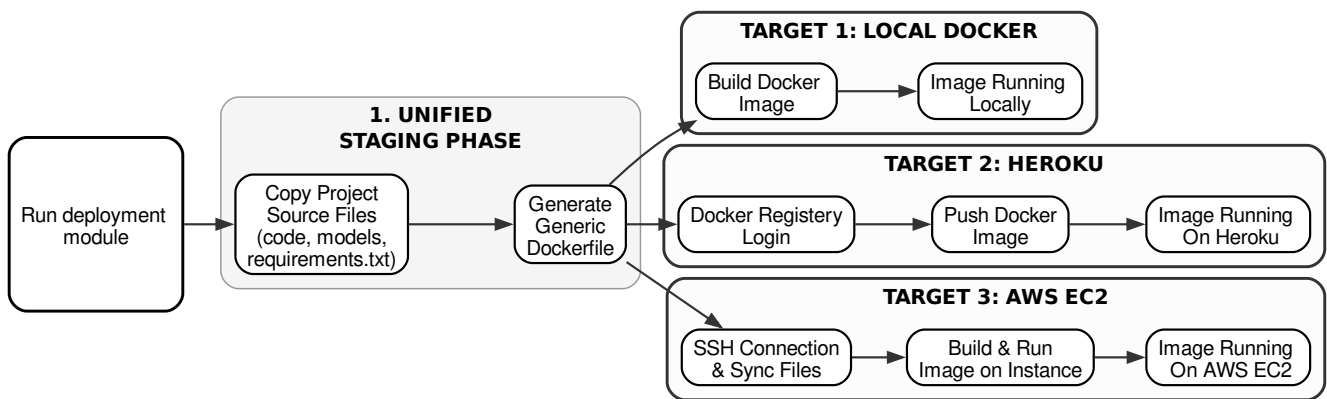


Figure 4.1: Deployment Orchestration Workflow

4.6 Model Version Control (VCS)

The internal VCS provides reproducibility for deployed pipelines. It calculates a stable SHA-1 hash for each commit based on its timestamp and commit message. Models and configurations are archived into an `experiments/` subdirectory. The system maintains a global index (`experiments.json`) containing parent-child commit histories, HEAD references, and active deployed markers, facilitating rollbacks and deployments of historical model versions.

The version control subsystem is implemented in `vcs.py`:

- `calculate_hash(timestamp, message)`: Computes the 7-character SHA-1 hash value that acts as the commit identifier.
- `init(args)`: Configures the repository workspace by creating directories and saving the default index (`experiments.json`).
- `commit(args)`: Validates that models exist, generates a commit hash, copies the active configuration and models to `experiments/<hash>/`, and appends the commit entry to the registry.
- `log(args)`: Lists all recorded commits, showing details such as commit hash, creation date, message, and deploy status.
- `show(args)`: Displays the details and settings of a target commit.
- `deploy(args)`: Deploys a specific commit by copying its stored model files and configuration back into the active runtime workspace.

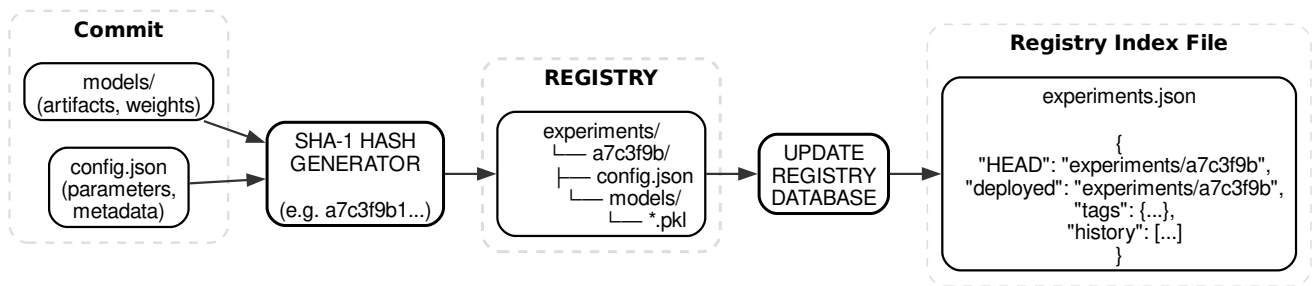


Figure 4.2: VCS Commit Workflow

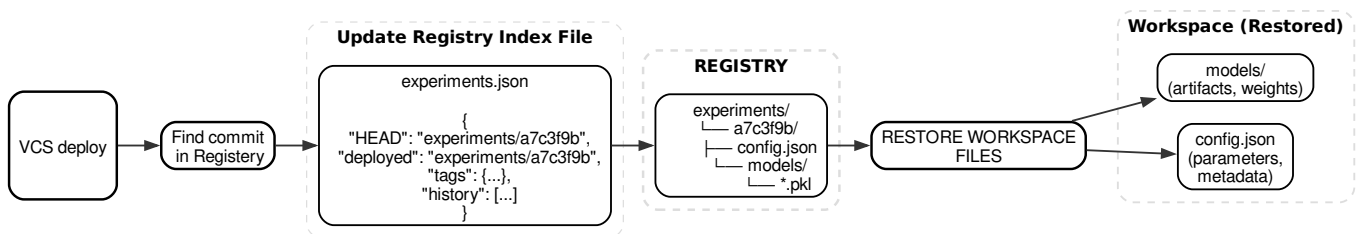


Figure 4.3: VCS Deploy Workflow

4.7 Command-Line Interface and CLI Commands

The orchestrator and version control features are exposed to the user through command-line interfaces inside `deployment.py` and `vcs.py`. The supported commands and their behaviors are:

Orchestrator Commands

- **prepare:** Prepares the deployment staging area. It copies service modules (e.g. `server.py`, `modelservice.py`) and the `accelera` source library to the `accelera_deployment` folder, and generates the serving `Dockerfile` and `requirements.txt`.
- **build:** Builds the local Docker container. It invokes local subprocess shell commands (e.g. `docker build -t <image_name> .`) with an optional `-no-cache` flag.
- **run-local:** Starts the local Docker container. It stops and removes any conflicting containers on the port before launching the container using port mappings.
- **local:** Orchestrates the complete local container pipeline by sequentially running the **prepare**, **build**, and **run-local** steps.
- **heroku-login:** Logs the client environment into the Heroku CLI and Heroku Docker registry.

- **heroku-deploy**: Packages and deploys the container to Heroku. It triggers login, pushes the Docker image to Heroku's registry, and releases the container for traffic.
- **heroku-push** and **heroku-release**: Granular command calls that push the container image and release it separately.
- **ec2-deploy**: Deploys to a remote AWS EC2 instance. It establishes an SSH connection, synchronizes files using **rsync**, compiles/builds the Docker container on the host, runs it, and probes local health targets.
- **ec2-status**, **ec2-logs**, and **ec2-stop**: Operational tools to verify container execution, read Uvicorn outputs, and terminate the container on the target EC2 node.

Version Control Commands

- **init**: Sets up a new model version control workspace by initializing directories and registries.
- **commit**: Snapshots active model parameters and configurations under a message flag (**-m**).
- **log**: Prints the history log of all model commits.
- **show**: Prints the configuration dictionary details of a target commit.
- **deploy**: Restores a specified commit back into the staging area for subsequent deployment execution.

4.8 Design Constraints and Challenges

The design of the Deployment module is subject to several key constraints:

- **Package Size Optimization**: To minimize container build times and storage footprints, heavy libraries like PyTorch or TensorFlow are excluded from the serving runtime unless explicitly required by the model.
- **Non-Interactive Execution**: Staging and EC2 orchestration must run headlessly in CI/CD pipelines, necessitating automatic Docker installs and automated SSH host key acceptance (**StrictHostKeyChecking=accept-new**).
- **I/O Isolation**: Process and SSH connection errors are caught as base **OSError** types to provide descriptive feedback on firewall and network issues without interrupting local execution.

During development, Mazen addressed three major implementation challenges:

1. **Great Expectations Init Latency**: Creating the GX validation context on every request inflated latencies by 1.2 seconds. Mazen solved this by instantiating the context only once during class initialization in **InputSchema** and validating incoming payloads in memory.

2. **EC2 Port Binding Lockups:** SSH commands occasionally hung if the remote port was bound to a zombie process. Mazen resolved this by adding a pre-flight cleaning command (`fuser -k -n tcp <port>`) to force-release the target port before running the Docker container.
3. **Asynchronous Logging Race Conditions:** High-frequency concurrent requests caused line fragmentation in the telemetry logging file. Mazen resolved this by wrapping the write calls in an append loop using Python file locking (`fcntl.flock`), ensuring write atomicity.

Chapter 5: System Testing and Verification

5.1 Pipeline Verification

The parallel executor and voting strategies were verified using unit tests.

- **Parallel Execution Tests:** Verify that the topological execution order is preserved, independent branches execute concurrently, and GIL lock release/acquisition behaves correctly.
- **Merge Node Tests:** Validate the correctness of the C++ hard-voting implementation. The test cases verify that the argmax voting correctly matches SciPy baselines for 1D and 3D prediction arrays, and handles the single-model edge case.

5.2 Benchmark Subsystem Verification

The benchmark module tests check that the Node.js API endpoints and database operations behave correctly:

- **Authentication tests:** Check token generation, verify that invalid login details are rejected, and confirm that the signup route hashes passwords correctly.
- **Dashboard routing tests:** Validate user-dashboard calls and verify that admin-level routes reject normal user accounts.
- **Validation tests:** Verify that the server rejects submissions with incorrect row counts or invalid URL structures.

5.3 Deployment Module Verification

The verification of the Deployment module is performed through a comprehensive test suite managed by the `pytest` framework. To ensure high test isolation and allow headless execution in CI/CD pipelines, the test suite leverages extensive mocking via the Python `monkeypatch` utility to decouple tests from system-level configurations, active network services (e.g., Heroku API, AWS EC2 hosts), and runtime dependency installations.

Tests are executed inside temporary filesystem workspaces generated dynamically using the `tmp_path` fixture. This ensures that the generated version control indices, log files, and Docker deployment artifacts do not contaminate the primary workspace.

5.3.1 Subsystem Verification Details

The verification targets the six major components of the deployment module:

1. **Model Version Control (VCS) verification:** Validates the stability of the 7-character short SHA-1 hash calculation, confirming that identical timestamp-message pairs produce identical commit IDs. Tests ensure that `vcs.init()` initializes the

experiments metadata directory structure and saves a clean registry template (`experiments.json`). Commit operations are verified to correctly copy model serialization binaries (e.g., `model.pkl`), snapshot active configurations, update the `HEAD` reference, and append metadata entries. Rollback capabilities are verified by deploying historical commits and confirming that the active runtime files are updated with the corresponding commit snapshots.

2. **Model Service Pipeline verification:** Tests check that the `ModelService` correctly parses configurations, loads active model estimators and preprocessors in the correct sequence, and performs inference. Mock tests confirm that preprocessing steps (such as scaling or encoding) are applied in order before predictions are generated.
3. **Runtime Schema Validation verification:** Validates the runtime behavior of input schema checking. Ephemeral Great Expectations suites are mocked to verify column-presence verification (identifying missing or unexpected fields) and value-bound checking. The data type auto-coercion system is verified by testing how it handles numeric conversions (casting strings to floats or integers) and parsing boolean representations (such as `"true"`, `"false"`, or numeric binary flags).
4. **Web Serving Endpoints verification:** Verifies that the FastAPI application correctly routes incoming HTTP requests. Tests evaluate the `/predict` and `/predict/csv` endpoints for JSON inputs and multi-row CSV file uploads. Endpoint testing checks that success responses return correct JSON structures with HTTP 200 codes, validation errors throw HTTP 422 codes, and server errors return HTTP 500 codes.
5. **Prediction Tracking and Telemetry verification:** Verifies the telemetry logging behavior. Tests ensure that when telemetry is disabled, no disk writes occur. When active, request parameters, model outputs, timestamps, and latency figures are verified to be written as valid JSON lines to the log file. Additionally, tests verify the telemetry reader's ability to aggregate counts (successful runs, error runs, total row counts) and report them dynamically.
6. **Orchestration CLI verification:** Validates port parsing limits (allowing ports between 1 and 65535, while rejecting negative numbers, non-numeric values, or out-of-bound ports). Tests check the preparation task (`prepare`) to verify that it creates a clean staging directory, compiles a standard `Dockerfile` using a Python slim base, and builds a clean `requirements.txt` containing only runtime-specific dependencies.

5.3.2 Deployment Test Results

When executed in an environment with all core dependencies installed, all 51 test cases compile and pass successfully. Table 5.1 summarizes the verification results for the deployment module.

Table 5.1: Deployment Module Test Suite Verification Summary

Test Component File	Test Cases	Passed	Failed	Verified Feature
vcs_test.py	14	14	0	Registry snapshots, workspace rolld
modelservice_test.py	6	6	0	Sequencing of preprocessor transfor
schema_validation_test.py	8	8	0	Data coercion and column checking
server_test.py	9	9	0	FastAPI routing, telemetry integra
tracking_test.py	3	3	0	Telemetry logging and count aggreg
deployment_test.py	11	11	0	Staging setup, Dockerfile compiling
Total	51	51	0	End-to-End System Correctne

This successful validation confirms that the deployment engine operates as intended, offering robust models, automated runtime validation, telemetry logging, and repeatable container deployments without regression errors.

Chapter 6: Conclusions and Future Work

6.1 Summary of Contributions

In this graduation project, Mazen Adel designed, implemented, and verified key subsystems for Accelera:

- **Pipeline Execution Layer:** A concurrent C++ scheduler utilizing semaphores for hardware thread limitation, safety gates for GPU tasks, and active consumer counting for memory release, paired with numpy/scipy-based majority voting nodes.
- **Benchmark Subsystem:** A Node.js Express backend and MongoDB schema model supporting JWT/bcrypt authentication and a React frontend for task management, dashboards, and leaderboard display.
- **Serving and Deployment Infrastructure:** A production-ready FastAPI model server featuring Great Expectations schema validation, async request tracking, model versioning (VCS) hash snapshotting, and automated scripts for Local Docker, Heroku, and remote AWS EC2 deployment targets.

6.2 Future Extensions

Future improvements can focus on the following:

- **Horizontal Scheduling:** Extend the pipeline parallel scheduler to distribute graph branches across different cluster nodes instead of limiting execution to a single host.
- **Robust Encryption:** Improve data security in the benchmark module by encrypting ground truth files on MongoDB and verifying predictions inside sandbox runners.
- **Serverless Serving Targets:** Add serverless deployment capabilities (such as AWS Lambda) to the orchestrator to deploy lightweight models without maintaining active virtual servers.